

Lab 1:

The screenshot shows pgAdmin 4 with two sessions connected to a PostgreSQL 18 database named 'dvdrental'. Session A (left) is executing a query with the following steps:

```
1 -- Step 1
2 BEGIN;
3 UPDATE film SET rental_rate = 4.99
4 WHERE film_id = 1;
5 -- Holds X-Lock on film_id = 1
6
7 --Step 3 (Run after Step 2 hangs)
8 COMMIT;
9 -- Releases the lock
```

The Data Output tab shows: UPDATE 1. Query returned successfully in 80 msec.

Session B (right) is idle, waiting for Session A to complete. Its Data Output tab shows: Query returned successfully in 80 msec.

A green status bar at the bottom right indicates: Query returned successfully in 80 msec.

Step 1: Session A updates film_id = 1 and holds an exclusive lock on the row.

The screenshot shows the same pgAdmin 4 interface. Session A (left) is still executing the same query as in Step 1.

Session B (right) has a new query in its Query History:

```
1 -- Step 2
2 BEGIN;
3 UPDATE film SET rental_rate = 5.99
4 WHERE film_id = 1;
5 -- BLOCKS IMMEDIATELY!
6 -- Waits for Session A
7
8 -- Step 4 (Observing)
9 -- Executes automatically after Acommits.
10 COMMIT;
```

The Data Output tab for Session B shows: UPDATE 1. Query returned successfully in 80 msec.

A status bar at the bottom right indicates: Waiting for the query to complete... 00:00:01.464.

Step 2: Session B tries to update the same row and is blocked, waiting for Session A.

The screenshot shows the pgAdmin 4 interface with two parallel SQL sessions. The left session, labeled 'Step 1', contains the following SQL code:

```

1  -- Step 1
2  BEGIN;
3  UPDATE film SET rental_rate = 4.99
4  WHERE film_id = 1;
5  -- Holds X-Lock on film_id = 1
6
7  --Step 3 (Run after Step 2 hangs)
8  COMMIT;
9  -- Releases the lock

```

The right session, labeled 'Step 2', contains the following SQL code:

```

1  -- Step 2
2  BEGIN;
3  UPDATE film SET rental_rate = 5.99
4  WHERE film_id = 1;
5  -- BLOCKS IMMEDIATELY!
6  -- Waits for Session A
7
8  -- Step 4 (Observing)
9  -- Executes automatically after Acommits.
10 COMMIT;

```

Both sessions show a 'Data Output' section with the message 'Query returned successfully in 123 msec.' and 'Query complete 00:00:00.123'.

Step 3: Session A commits and releases the lock.

The screenshot shows the pgAdmin 4 interface with two parallel SQL sessions. The left session, labeled 'Step 1', contains the following SQL code:

```

1  -- Step 1
2  BEGIN;
3  UPDATE film SET rental_rate = 4.99
4  WHERE film_id = 1;
5  -- Holds X-Lock on film_id = 1
6
7  --Step 3 (Run after Step 2 hangs)
8  COMMIT;
9  -- Releases the lock

```

The right session, labeled 'Step 2', contains the following SQL code:

```

1  -- Step 2
2  BEGIN;
3  UPDATE film SET rental_rate = 5.99
4  WHERE film_id = 1;
5  -- BLOCKS IMMEDIATELY!
6  -- Waits for Session A
7
8  -- Step 4 (Observing)
9  -- Executes automatically after Acommits.
10 COMMIT;

```

Both sessions show a 'Data Output' section with the message 'Query returned successfully in 75 msec.' and 'Query complete 00:00:00.075'.

Step 4: Session B continues automatically after the lock is released and then commits.

Lab 2:

pgAdmin 4

File Object Tools Edit View Window Help

dvdrental/postgres@PostgreSQL 18*

Query Query History

```
1 -- Step 1
2 BEGIN;
3 SELECT rental_rate FROM film
4 WHERE film_id = 10;
5 -- Result: 4.99 (S-Lock released)
6
7 --Step 3
8 SELECT rental_rate FROM film
9 WHERE film_id = 10;
10 -- NEWRESULT: 0.99!
11 COMMIT;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

rentalRate	numeric(4,2)
1	0.99

Total rows: 1 Query complete 00:00:00.136 CRLF Ln 1, Col 1

Step 1: The first transaction reads the rental rate as 4.99.

pgAdmin 4

File Object Tools Edit View Window Help

dvdrental/postgres@PostgreSQL 18*

Query Query History

```
1 -- Step 2
2 BEGIN;
3 UPDATE film SET rental_rate = 0.99
4 WHERE film_id = 10;
5
6 COMMIT;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

COMMIT
COMMIT

Total rows: 1 Query complete 00:00:00.092 CRLF Ln 6, Col 1

Step 2: The second transaction updates the rental rate to 0.99 and commits the change.

The image displays two screenshots of the pgAdmin 4 interface, illustrating a database transaction. Both screenshots show the 'dvdrental' database connected to a PostgreSQL 18 server.

Left Screenshot (Initial State):

- Query:**

```
1 -- Step 1
2 BEGIN;
3 SELECT rental_rate FROM film
4 WHERE film_id = 10;
5 -- Result: 4.99 (S-Lock released)
6
7 --Step 3
8 SELECT rental_rate FROM film
9 WHERE film_id = 10;
10 -- NEWRESULT: 0.99!
11 COMMIT;
```
- Data Output:** Shows a single row with `rental_rate` as `numeric (4,2)` and a value of `0.99`.
- Messages:** A green notification box states: "Successfully run. Total query runtime: 140 msec. 1 rows affected."

Right Screenshot (After Update):

- Query:**

```
1 -- Step 2
2 BEGIN;
3 UPDATE film SET rental_rate = 0.99
4 WHERE film_id = 10;
5
6 COMMIT;
```
- Data Output:** Shows the `COMMIT` message.
- Messages:** A green notification box states: "Query returned successfully in 92 msec."

The second screenshot shows the same transaction being re-run, demonstrating a non-repeatable read where the same query returns a different value (0.99) due to the update performed within the transaction.

Step 3: The same transaction reads 0.99, demonstrating a non-repeatable read.

Lab 3

The screenshot shows pgAdmin 4 with two sessions open. The left session, 'dvdrental/postgres@PostgreSQL 18', has a query window with the following SQL:

```
-- step 1
1 BEGIN;
2
3 UPDATE film SET rental_rate = 2.99
4 WHERE film_id = 100;
5
6 -- Step 3
7 UPDATE film SET rental_rate = 3.99
8 WHERE film_id = 200;
9 -- Blocks (Waiting for B)
```

The right session, 'dvdrental/postgres@PostgreSQL 18', has a query window with the following SQL:

```
-- Step 2
1 BEGIN;
2
3 UPDATE film SET rental_rate = 2.99
4 WHERE film_id = 200;
5
6 -- Step 4
7 UPDATE film SET rental_rate = 3.99
8 WHERE film_id = 100;
9 -- BOOM! ERROR: deadlock detected
```

Both sessions show a 'Data Output' tab with the message: 'Query returned successfully in 70 msec.' and 'Query returned successfully in 73 msec.' respectively. A green box at the bottom right indicates 'Query returned successfully'.

Step 1: Session A locks film_id = 100 and then tries to update film_id = 200.
Step 2: Session B locks film_id = 200 and then tries to update film_id = 100.

The screenshot shows the same pgAdmin 4 interface. Session A's query window now shows the following SQL:

```
-- step 1
1 BEGIN;
2
3 UPDATE film SET rental_rate = 2.99
4 WHERE film_id = 100;
5
6 -- Step 3
7 UPDATE film SET rental_rate = 3.99
8 WHERE film_id = 200;
9 -- Blocks (Waiting for B)
```

Session B's query window remains the same as in the previous screenshot. The 'Data Output' tab for Session A now shows: 'Waiting for the query to complete... 00:00:01.765'. The status bar at the bottom indicates 'Total rows: Waiting for the query to complete... 00:00:01.765'.

Step 3: Session A is blocked because Session B holds the required lock.

pgAdmin 4

File Object Tools Edit View Window Help

Servers (1)
PostgreSQL 18
Databases (2)
dvdrental

Query Query History

```

1 -- Step 1
2 BEGIN;
3 UPDATE film SET rental_rate = 2.99
4 WHERE film_id = 100;
5
6 -- Step 3
7 UPDATE film SET rental_rate = 3.99
8 WHERE film_id = 200;
9 -- Blocks (Waiting for B)

```

Data Output Messages Notifications

UPDATE 1
Query returned successfully in 12 secs 643 msec.

Query complete 00:00:12.643 CRLF Ln 7, Col 1

dvdrental/postgres@PostgreSQL 18*

Query Query History

```

1 -- Step 2
2 BEGIN;
3 UPDATE film SET rental_rate = 2.99
4 WHERE film_id = 200;
5
6 -- Step 4
7 UPDATE film SET rental_rate = 3.99
8 WHERE film_id = 100;
9 -- BOOM! ERROR: deadlock detected

```

Data Output Messages Notifications

ERROR: deadlock detected
Process 18752 waits for ShareLock on transaction 1020; blocked by process 2264.
Process 2264 waits for ShareLock on transaction 1021; blocked by process 18752.

SQL state: 40P01
Detail: Process 18752 waits for ShareLock on transaction 1020; blocked by process 2264.
Process 2264 waits for ShareLock on transaction 1021; blocked by process 18752.
Hint: See server log for query details.
Context: while locking tuple (5,12) in relation "film"

Query complete 00:00:01.063 CRLF Ln 7, Col 1

✓ Query returned successfully in 12 secs 643 msec. ✕

Step 4: PostgreSQL detects the deadlock and rolls back Session B, allowing Session A to complete.

Lab 4:

pgAdmin 4

File Object Tools Edit View Window Help

Servers (1)
PostgreSQL 18
Databases (2)
dvdrental

Query Query History

```

1 -- Step 1
2 BEGIN;
3 SELECT * FROM inventory
4 WHERE inventory_id = 10 FOR UPDATE;
5 -- Locks the row proactively
6
7 -- Step 3
8 INSERT INTO rental(rental_date, inventory_id , customer_id)
9 VALUES (NOW() , 10, 1, 1);
10 COMMIT;

```

Data Output Messages Notifications

Showing rows: 1 to 1

inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
1	10	2	2006-02-15 10:09:17

Query complete 00:00:00.232 CRLF Ln 1, Col 1

dvdrental/postgres@PostgreSQL 18*

Query Query History

```

1 -- Step 2
2 BEGIN;
3 SELECT * FROM inventory
4 WHERE inventory_id = 10 FOR UPDATE;
5 -- BLOCKS IMMEDIATELY
6
7 -- Step 4
8 -- Unblocks after A commits.
9 -- App logic detects it 's already rented!
10 ROLLBACK;

```

Data Output Messages Notifications

Successfully run. Total query runtime: 232 msec. 1 rows affected. ✕

Step 1: Session A locks the selected inventory row using FOR UPDATE.

The screenshot shows pgAdmin 4 with two sessions. Session A (left) has executed the following SQL:

```
-- Step 1
BEGIN;
SELECT * FROM inventory
WHERE inventory_id = 10 FOR UPDATE;
-- Locks the row proactively
```

The Data Output tab shows a single row from the inventory table:

inventory_id [PK] integer	film_id smallint	store_id smallint	last_update timestamp without time zone
10	2	2	2006-02-15 10:09:17

Session B (right) has executed the following SQL:

```
-- Step 2
BEGIN;
SELECT * FROM inventory
WHERE inventory_id = 10 FOR UPDATE;
-- BLOCKS IMMEDIATELY
```

The Data Output tab for Session B shows the message: "Waiting for the query to complete... 00:00:01.867".

Step 2: Session B waits because the same row is already locked by Session A.

The screenshot shows the same pgAdmin 4 interface. Session A (left) has executed the following SQL:

```
-- Step 3
INSERT INTO rental(rental_date, inventory_id, customer_id)
VALUES (NOW(), 10, 1, 1);
COMMIT;
```

The Data Output tab shows the message: "ROLLBACK" and "Query returned successfully in 94 msec."

Session B (right) has executed the following SQL:

```
-- Step 4
-- Unblocks after A commits.
-- App logic detects it 's already rented!
ROLLBACK;
```

The Data Output tab for Session B shows the message: "Query returned successfully in 94 msec."

Step 3: Session A inserts the rental record successfully.

pgAdmin 4

File Object Tools Edit View Window Help

dvdrental/postgres... x dvdrental/postgres@PostgreSQL 18*

dvdrental/postgres... x dvdrental/postgres@PostgreSQL 18*

Servers (1)
PostgreSQL 18
Databases (2)
dvdrental

- > Casts
- > Catalogs
- > Event Triggers
- > Extensions
- > Foreign Data W
- > Languages
- > Publications
- > Schemas
- > Subscriptions
- > postgres
- > Login/Group Roles
- > Tablespaces

Query Query History

```
1 -- Step 1
2 BEGIN;
3 SELECT * FROM inventory
4 WHERE inventory_id = 10 FOR UPDATE;
5 -- Locks the row proactively
6
7 -- Step 3
8 INSERT INTO rental(rental_date, inventory_id , customer
9 VALUES (NOW() , 10, 1, 1);
10 COMMIT;
```

Data Output Messages Notifications

ROLLBACK

Query returned successfully in 94 msec.

Total rows: Query complete 00:00:00.094 CRLF Ln 10, Col 1

Query Query History

```
1 -- Step 2
2 BEGIN;
3 SELECT * FROM inventory
4 WHERE inventory_id = 10 FOR UPDATE;
5 -- BLOCKS IMMEDIATELY
6
7 -- Step 4
8 -- Unblocks after A commits.
9 -- App logic detects it 's already rented!
10 ROLLBACK;
```

Data Output Messages Notifications

ROLLBACK

Query returned successfully in 109 msec.

Total rows: Query complete 00:00:00.109 CRLF Ln 10, Col 1

Step 4: After A commits, B can continue, showing that the lock prevents double booking.